Todo Mecánica Express — Plan maestro del proyecto

Versión 0.2

11 de octubre de 2026

Borrador de trabajo para revisión con el taller

1 Contexto y objetivo

Todo Mecánica Express es un taller mecánico ubicado en Camino vecinal, parcela 10, Barrancas, San Felipe, Región de Valparaíso. El taller mantiene presencia en Instagram y TikTok.

El proyecto busca construir dos experiencias conectadas:

- **Sitio público:** presentar el taller, publicar información validada sobre sus servicios y trabajos, facilitar el contacto y, cuando se defina el proceso, recibir solicitudes.
- **Sistema interno:** ordenar solicitudes, agenda y operación diaria; avanzar hacia diagnósticos, cotizaciones, órdenes de trabajo e historial solo cuando el taller valide cada etapa.

El desarrollo es evolutivo y no se detiene a la espera del feedback del jefe. Este plan es la referencia principal de trabajo: permite construir prototipos desde ahora en ramas temporales. Los patrones investigados en otros talleres mecánicos sirven como hipótesis de diseño, no como requisitos confirmados. Cuando llegue feedback, se usa para refinar o descartar funciones en las ramas de trabajo. `main` conserva únicamente las versiones que el equipo ha aprobado como listas para mostrar al jefe.

1.1 Estado conocido al preparar este informe

- **Confirmado según la información disponible:** el `main` remoto descrito en el informe contiene el README y aún no incluye la aplicación. En el PC principal se indicó una aplicación Next.js local en `web/`; la vista compartida mostraba una portada con el nombre, ubicación y enlace a Instagram. No se confirmó aquí el contenido completo, el commit ni el envío de esa copia local.
- **Confirmado:** se indicó desarrollar inicialmente en `version/0.1-sitio-publico`; antes de publicar cambios, comprobar la rama con `git status -sb`.
- **Pendiente de validar con el taller:** servicios, horarios, teléfono, WhatsApp, fotografías, precios y capacidad de atención. Este informe no los da por establecidos.

1.2 Cómo leer los estados

Confirmado
: Información respaldada por el responsable, el repositorio o una vista local identificable. Se debe indicar la fuente cuando se actualice este informe.

Propuesto
: Alternativa de diseño o técnica que todavía puede cambiar; no es un requisito aprobado.

Pendiente
: Dato o decisión que requiere conversación y validación con el taller.

2 Alcance por etapas

Las etapas son una hoja de ruta y una referencia para avanzar aunque todavía no haya feedback. En cada una se distingue lo construido y confirmado según la evidencia disponible, las propuestas basadas en patrones observados o investigados en talleres mecánicos (hipótesis de diseño) y lo que falta validar con Todo Mecánica Express. Ninguna propuesta se presenta como una función ya implementada o aprobada.

V0.1 — Prototipo público local (en curso)

Construido: según la vista disponible, una portada local con el nombre, la ubicación y un enlace a Instagram; se indicó que está en una aplicación Next.js local, no verificada por completo aquí.

Propuesto (hipótesis): una portada adaptable, clara y útil como punto de entrada, siguiendo patrones habituales de presentación pública en talleres.

Por validar con Todo Mecánica Express: identidad visual, textos, recursos autorizados y si la portada representa correctamente al taller.

V1 — Captación

Construido: sirve de base el prototipo local V0.1 descrito anteriormente; no se ha confirmado un sitio V1 completo.

Propuesto (hipótesis): presentar servicios confirmados y ofrecer vías de contacto sencillas, siguiendo patrones de sitios de talleres; el método concreto de recepción puede definirse sin asumir que ya existe un formulario.

Por validar con Todo Mecánica Express: servicios, horarios, teléfono o WhatsApp, fotos, canal de recepción y aprobación del contenido antes de publicar.

V2 — Solicitudes y agenda

Construido: no se ha confirmado un formulario, bandeja interna ni agenda implementados.

Propuesto (hipótesis): registrar solicitudes, organizarlas en una bandeja y dejar que el personal confirme, rechace o proponga otra hora; la solicitud no equivale a una cita confirmada.

Por validar con Todo Mecánica Express: quién responde, datos mínimos, disponibilidad, capacidad por bloque y cómo evitar duplicidades.

V3 — Operación

Construido: no se ha confirmado un módulo operativo implementado.

Propuesto (hipótesis): abrir una orden o ficha al recibir el vehículo; registrar después el diagnóstico y la cotización, y ejecutar la reparación solo tras la aprobación explícita del cliente.

Por validar con Todo Mecánica Express: proceso real de recepción, campos y responsables, registro de autorización y manejo de cambios en el trabajo cotizado.

V4 — Seguimiento

Construido: no se ha confirmado historial, avisos ni informes implementados.

Propuesto (hipótesis): conservar un historial de atenciones y añadir avisos o informes solo si patrones de uso y necesidades reales los justifican.

Por validar con Todo Mecánica Express: qué información es útil, quién puede consultarla, qué avisos se necesitan y qué indicadores ayudan a tomar decisiones.

V5 — Expansión opcional

Construido: no se ha confirmado inventario, pagos, facturación ni integraciones implementados.

Propuesto (hipótesis): considerar esas funciones como opciones, no como parte comprometida del producto, y prototiparlas solo si resuelven una necesidad identificada.

Por validar con Todo Mecánica Express: necesidad, procesos, costos, responsables y condiciones aplicables antes de desarrollar.

3 Arquitectura propuesta

Propuesto: una aplicación web con Next.js, React y TypeScript. El sitio público y el futuro panel del taller vivirían en `web/`, con navegación, permisos y objetivos diferenciados. CSS o Tailwind pueden apoyar una interfaz adaptable. Para V2 se propone PostgreSQL alojado en Supabase, con autenticación del personal y políticas de acceso. La plataforma de alojamiento y la elección definitiva se decidirán al preparar una publicación.

3.1 Flujo operativo y conceptos

Los siguientes conceptos son distintos y no deben tratarse como sinónimos:

1. Una **solicitud** expresa interés o describe una necesidad; por sí sola no reserva una hora.
2. Una **cita confirmada** existe cuando el taller valida fecha y capacidad y confirma la hora al cliente.
3. La **recepción** registra la llegada del vehículo. Al recibirlo, puede abrirse una **orden o ficha de trabajo** para identificar el vehículo y dejar constancia de la atención.
4. En esa ficha se registra el **diagnóstico** y se prepara la **cotización** correspondiente.
5. La reparación cubierta por la cotización solo se ejecuta después de la **aprobación explícita del cliente**. Si el alcance cambia, se solicita la aprobación que corresponda antes de realizar el trabajo adicional.
6. Al finalizar, se registra la entrega y, si el taller valida esa función, se incorpora la atención al historial del vehículo.

Para fallas de duración desconocida, la cita inicial puede tener como propósito el diagnóstico. No se debe prometer de antemano el alcance ni el precio de la reparación.

4 Organización de archivos

4.1 Estructura actual conocida

La información disponible describe esta estructura; no se han inspeccionado todos los archivos de la aplicación local:

```
Todo-Mecanica-Express/
|-- README.md               (descrito como presente en main remoto)
‘-- web/                    (aplicacion local indicada)
    ‘-- src/app/page.tsx    (portada mostrada en la vista local)
```

4.2 Destino propuesto

El árbol siguiente es una guía para el crecimiento futuro. No exige crear carpetas vacías ni afirma que estas rutas existan hoy:

```
Todo-Mecanica-Express/
|-- README.md
|-- docs/
|   |-- plan-maestro.md
|   |-- flujo-del-taller.md
|   |-- decisiones.md
|   ‘-- versiones/
‘-- web/
```

```
|-- public/                       (recursos autorizados)
|-- src/
|   |-- app/
|   |   |-- page.tsx              (portada)
|   |   |-- servicios/page.tsx     (catalogo confirmado, V1)
|   |   |-- contacto/page.tsx      (contacto o solicitud, V1-V2)
|   |   |-- panel/                  (acceso interno, V2+)
|   |   `-- api/                    (rutas necesarias, V2+)
|   |-- components/                 (interfaz compartida)
|   |-- lib/                        (validacion y reglas)
|   `-- types/                      (tipos del dominio)
`-- supabase/migrations/           (si se adopta Supabase)
```

`app/` organiza páginas y rutas; `components/` contiene interfaz reutilizable; `lib/` concentra validaciones y reglas; la base de datos conserva información persistente. Las rutas del panel deberán comprobar sesión y permisos en el servidor. Estas responsabilidades son propuestas y se concretarán al implementar cada etapa.

5 Datos previstos para V2 y V3

El siguiente esquema es **propuesto**; los campos, relaciones y procesos deben validarse antes de implementarlos:

Cliente
Nombre, contacto y canal preferido; puede tener varios vehículos.

Vehículo
Patente cuando corresponda, marca/modelo y vínculo con el cliente; puede tener varias solicitudes y atenciones.

Solicitud
Motivo, servicio consultado, canal, fecha deseada y estado; puede originar una cita.

Cita
Fecha y hora, duración o bloque, estado y responsable; se vincula con la solicitud y el vehículo.

Recepción
Ingreso, observaciones y evidencia autorizada; registra la llegada del vehículo.

Orden o ficha de trabajo
Identificación de la atención y registro de diagnóstico, cotización, autorizaciones, actividades, responsables, estados y fechas.

Cotización
Ítems, montos, vigencia y respuesta del cliente; la reparación cubierta requiere aprobación antes de ejecutarse.

Estados iniciales sugeridos, pendientes de validación: solicitud: nueva, en revisión, confirmada, rechazada o cancelada; cita: confirmada, atendida, no asistió o cancelada. Nombres, transiciones y permisos se acordarán con el taller. Las fechas y horas deben almacenarse con una política consistente para Chile.

6 Seguridad y calidad

- El sitio público debe funcionar en teléfono y computador, cargar con rapidez y describir únicamente servicios reales y confirmados.

- El futuro panel requiere cuentas del personal y permisos por función. Visitantes no deben acceder a datos de clientes, vehículos ni trabajo interno.
- Los formularios deben validar en el servidor, limitar envíos automatizados y recolectar solo los datos necesarios.
- Las credenciales deben permanecer fuera del repositorio; tampoco se deben exponer datos reales de clientes en ejemplos públicos.
- Antes del uso operativo hacen falta copias de seguridad, recuperación, manejo de errores y pruebas del flujo de solicitud y confirmación.
- No automatizar mensajes, cobros ni reservas irrevocables hasta definir quién responde, en qué horario y con qué capacidad.

7 Git, versiones y ciclo de feedback

Regla de integración: `main` representa la versión que el equipo aprobó para mostrar al jefe. La falta de respuesta o feedback no bloquea el desarrollo: se continúa prototipando y refinando en ramas temporales, dejando explícitas las hipótesis. Cuando llegue feedback, se refinan o descartan las funciones en esas ramas. Solo se integra a `main` una versión revisada y aprobada internamente para presentarla; las ramas temporales se eliminan después de integrar o descartar su trabajo.

1. Partir de `main`, como referencia de la última versión aprobada internamente para mostrar al jefe.
2. Crear una rama `version/*` para cada entrega y usar ramas `feature/*`, `feedback/*`, `fix/*` o `docs/*` cuando ayuden a organizar cambios concretos.
3. Registrar en `docs/versiones/` el alcance, criterios de aceptación, pruebas, hipótesis y feedback recibido.
4. Desarrollar sin esperar feedback; cuando llegue, aplicar los comentarios o descartar las funciones correspondientes en las ramas de trabajo.
5. Revisar internamente cada entrega; integrar a `main` solo la versión aprobada para mostrar al jefe y borrar las ramas temporales integradas o descartadas.

Al proponer trabajo, Prism debe indicar la etapa, el requisito, los archivos afectados, el criterio de aceptación y qué se encuentra confirmado, propuesto o pendiente. Una idea escrita aquí no se considera aprobada por el taller.

8 Preguntas para la primera reunión

1. ¿Qué servicios ofrecen actualmente y cuáles requieren diagnóstico previo?
2. ¿Cuáles son el horario, el teléfono o WhatsApp oficial, los días de atención y los tiempos habituales?
3. ¿Quién recibe consultas y quién puede confirmar citas o aprobar cotizaciones?
4. ¿Cuántos vehículos pueden recibir por bloque y qué recursos limitan esa capacidad?
5. ¿Cómo registran hoy clientes, vehículos, fallas, cotizaciones, autorizaciones y entregas?
6. ¿Qué fotografías, logotipo, reseñas o trabajos se pueden publicar con autorización?
7. ¿Qué problema consume más tiempo: consultas, agenda, seguimiento, presupuestos u otro?
8. ¿Qué necesita ver el jefe para aprobar V1 y qué espera del panel de V2?

9 Registro de decisiones

Adoptada para el prototipo
Mantener un repositorio con `web/` para el sitio y la futura aplicación.

Adoptada para V0.1, según la información disponible
Next.js, React, TypeScript y Tailwind; se indicó que la aplicación local se creó con esa base. No implica que todos sus archivos se hayan verificado aquí.

Propuesta de dominio
Separar solicitud, cita, recepción y orden o ficha de trabajo para no confundir consultas, atención ni autorizaciones.

Propuesta técnica
Usar Supabase/PostgreSQL para V2; no se ha confirmado una base de datos configurada ni datos reales.

Pendiente
Validar marca, textos, servicios, agenda y despliegue según la operación y aprobación del taller.

10 Mantenimiento del informe

Revisar el documento después de cada conversación con el taller. Cambiar «Pendiente» por «Confirmado» solo cuando exista una fuente concreta (persona, fecha y decisión); registrar desacuerdos y nuevas versiones. Cuando el equipo apruebe el contenido, se podrá copiar a `docs/plan-maestro.md` en una rama de trabajo y actualizar allí el README si corresponde. El informe guía la implementación; el código y las pruebas describen lo que realmente existe.

10.1 Referencias de trabajo

- README del repositorio, consultado el 11 de octubre de 2026.
- Documentación de Next.js, para estructura de rutas y componentes.
- Guía Next.js + Supabase, como referencia para una integración futura.